

Scenario: “AutoFleet AI: The Autonomous Fleet Orchestration System Crisis”

Background:

AutoFleet AI Ltd. is developing a smart fleet management platform for electric and autonomous delivery vehicles. The system should predict maintenance, assign drivers autonomously, and optimize delivery routes based on real-time data.

Initially launched under an Agile MVP approach, the project spiraled into complexity as investor demands, regulatory changes, and technical misalignments collided with poor design governance.

Phase 1: Process Breakdown

- The Project Charter was never updated after Agile transition, so teams still follow SDLC documentation milestones.
- Sprint reviews produce incremental code, but no traceability to requirements.
- The QA team insists on Waterfall-style “final test phase,” causing repeated integration collapses every three sprints.
- The DevOps lead complains about unverified requirements being directly pushed to production.

Phase 2: Requirements Failure

- Stakeholders keep changing functional requirements: “driverless dispatch,” “predictive battery drain,” and “dynamic pricing.”
- User stories contradict each other:
 - “As a fleet manager, I want manual override controls so that I can take control of a vehicle.”
 - “As a system, I want no manual control to ensure safety compliance.”
- No acceptance criteria or MoSCoW prioritization exists and all features are declared “must have.”

Phase 3: Architecture Conflict

- Architects disagree:
 - Team X proposes an Event-Driven Architecture for real-time sensor streams and notifications.
 - Team Y wants a Layered Architecture with a centralized AI Decision Engine.
- No record of trade-off analysis or ADRs exists.
- The system accidentally achieved tight coupling between the route-planner and AI-predictor modules, breaking independent deployments.

Phase 4: Design and Code Violations

- A single class FleetController manages route optimization, charging logic, data logging, and UI control as a massive SRP violation.
- The Open/Closed Principle is broken and each time a new vehicle type is added (drone, truck, bike), core code is modified instead of extended.

- Developers misused Singletons for GPS services, creating race conditions across threads.
- A Factory Method for vehicle creation depends on hard-coded logic in controllers, not interfaces.

Phase 5: Behavioral Collapse

- The Observer pattern for vehicle health monitoring never unsubscribes listeners, flooding memory.
- The State pattern for vehicle lifecycle (Created → Dispatched → En-route → Delivered → Charging) is partially implemented, vehicles sometimes skip states.
- Command objects for “reroute” and “cancel delivery” fail in the undo queue, triggering false dispatches.

Phase 6: Documentation and Communication Breakdown

- UML diagrams show a monolithic architecture while the system is actually microservice-based.
- No C4 container views exist for microservices or external IoT feeds.
- Outdated README files describe Python APIs while half the services are now in Go.
- The lack of updated ADRs makes root-cause analysis impossible.

Your Challenge: 10 Advanced Scenario-Based Questions

1. Process Redesign: How would you align AutoFleet AI’s hybrid Waterfall-Agile workflow to restore traceability and iterative delivery? Include specific artifacts (e.g., updated Project Charter, Definition of Done).
2. Requirements Conflict Resolution: Propose a Requirements Engineering plan to capture, validate, and prioritize features using User Stories + Acceptance Criteria + MoSCoW prioritization.
3. Architectural Decision Reform: Write an ADR template comparing Event-Driven and Layered architectures for AutoFleet AI, including context, decision, and consequences.
4. Coupling and Cohesion Audit: Identify which modules demonstrate tight coupling and low cohesion. Propose refactoring using GRASP principles (Information Expert, Low Coupling, Controller).
5. SOLID Reconstruction: Refactor the FleetController class into SRP-compliant classes while maintaining OCP and DIP. Show how dependency injection can replace hard-coded vehicle types.
6. Creational Pattern Correction: Redesign the vehicle instantiation mechanism using an Abstract Factory or Builder pattern to handle new vehicle types without modifying existing code.
7. Behavioral Pattern Stabilization: Redefine the Observer and State implementations to prevent memory leaks and invalid transitions. Would a Mediator pattern help coordinate vehicle state changes?

8. C4 Model Recovery: How would you redraw the C4 Context, Container, and Component diagrams to reflect microservices, sensors, and AI modules accurately?
9. Documentation Strategy: Design a system-wide documentation recovery plan combining UML, ADRs, and README automation to keep docs in sync with code.
10. Continuous Improvement: Suggest a quality metric framework to evaluate AutoFleet AI's design maturity (cohesion, coupling, coverage, and architectural fitness over time).